

Herald — TELEGRAM

Herald — Telegram Channel Setup Guide

Table of contents

Pre-requisites

Step 1 — Create a bot via @BotFather

Step 2 — Get the chat ID

Option A — Direct messages (you [?] bot)

Option B — Group chat (bot in a group)

Option C — Channel (one-way broadcast)

Confirm the chat ID

Step 3 — Provide the credentials to Herald

Path A — Shell-export (recommended for personal workstation)

Path B — Project-local .env

Step 4 — Verify HealthCheck

Step 5 — Verify Send (E17)

Step 6 — (Optional) Verify Subscribe + Vertical Slice (E19)

Step 7 — (Future) Webhook ingress for production

Troubleshooting

Spec + code references

Herald — Telegram Channel Setup Guide

Field	Value
Status	LIVE (HRD-011 — code complete; awaiting operator credentials for E17/E19 live evidence)
Spec ref	V3 §11.1 + §32.2
HRD	HRD-011
Env vars	HERALD_TGRAM_BOT_TOKEN, HERALD_TGRAM_CHAT_ID, HERALD_TGRAM_LIVE_INBOUND (optional), HERALD_TGRAM_WEBHOOK_SECRET (reserved for future webhook ingress)
Code	commons_messaging/channels/ tgram/ (telebot.v3 vendored at submodules/telebot/ v3.3.8)

This guide walks you through every step needed to enable Telegram in Herald — from creating the bot to verifying the live integration test passes. Once you complete these steps, set the env vars in your shell or `.env`, hand the tokens back to the operator, and we will close HRD-011 with live E17/E19 evidence.

Table of contents

- [Pre-requisites](#)
- [Step 1 — Create a bot via @BotFather](#)
- [Step 2 — Get the chat ID](#)
- [Step 3 — Provide the credentials to Herald](#)
- [Step 4 — Verify HealthCheck](#)
- [Step 5 — Verify Send \(E17\)](#)
- [Step 6 — \(Optional\) Verify Subscribe + Vertical Slice \(E19\)](#)
- [Step 7 — \(Future\) Webhook ingress for production](#)
- [Troubleshooting](#)
- [Spec + code references](#)

Pre-requisites

- A Telegram account (mobile, desktop, or web client). The account is YOURS; the bot you create is a separate logical entity owned by your account.
- The Herald checkout at `/Users/milosvasic/Projects/Herald` (or wherever you cloned it).
- Either:
 - podman or docker available locally (for the quickstart compose stack), OR
 - a reachable Postgres instance per the §“Postgres” entries in `OPERATOR_CREDENTIALS.md`.

Step 1 — Create a bot via @BotFather

Telegram’s official “create-a-bot” interface is a bot called BotFather. You’ll chat with it to create your bot.

1. Open Telegram → tap the search icon → type BotFather.
2. Tap the result with the verified-blue-checkmark next to its username (the real one is @BotFather). **Don’t** tap any imposter accounts; they cannot create real bots and may phish you.
3. Start the chat with `/start`.
4. Send `/newbot` to BotFather.
5. BotFather asks for a display name. Pick something descriptive — examples:
 - Herald Operator Bot
 - Herald CI Notifications
 - MyProject Notify Bot

6. BotFather asks for a username. Must end in bot (or _bot). Examples:

- herald_operator_bot
- myproject_notify_bot

7. BotFather replies with a **bot token** of the form:

1234567890:AAH1lab2c3d4e5f6g7h8i9j0kK_LmN3oPqR4S5t

This token is the bot's password. Treat it like one — never paste it into a PR, a Slack channel, a commit message, or anywhere indexed.

8. **Copy the token immediately** and put it somewhere safe (password manager, secrets vault, secure note).

Token rotation: if you ever leak this token, message /revoke to BotFather and pick the bot to invalidate the old token + receive a new one. This is the only fix — git history scrubbing alone does NOT invalidate a leaked Telegram token.

Step 2 — Get the chat ID

Your bot needs to know **where** to send messages. That destination is identified by a numeric **chat ID** — a chat is a one-on-one DM, a group, or a channel.

Choose your chat type:

Option A — Direct messages (you [?] bot)

1. In Telegram, search for your bot's username (e.g. @herald_operator_bot).
2. Tap **Start** to open a DM with it.
3. Send any message (e.g. hello).
4. In a browser, visit:

<https://api.telegram.org/bot<YOUR-BOT-TOKEN>/getUpdates>

Replace <YOUR-BOT-TOKEN> with the real token from Step 1.

5. You'll see a JSON response. Inside it, locate:

```
"chat": { "id": 123456789, "first_name": "...", "type": "private" }
```

6. The id value (positive integer for DMs) is your chat ID.

Option B — Group chat (bot in a group)

1. Create a Telegram group (or use an existing one).
2. Add your bot to the group via the group's **Add Member** menu.

3. Send a message inside the group (any text).

4. In a browser, visit:

```
https://api.telegram.org/bot<YOUR-BOT-TOKEN>/getUpdates
```

5. Find the "chat": { "id": ..., "type": "supergroup" } block.

6. The id for groups is a **negative number** like -1001234567890.

Option C — Channel (one-way broadcast)

Channels are one-way: the bot can post but cannot read messages. Useful for notifications without two-way interaction.

1. Create a Telegram channel.

2. Add your bot as an **Administrator** (a regular member does NOT have post permission). Grant at least “Post Messages”.

3. Send any message to the channel (from your own account, since the bot is admin not subscriber).

4. In a browser, visit:

```
https://api.telegram.org/bot<YOUR-BOT-TOKEN>/getUpdates
```

5. Locate "chat": { "id": ..., "type": "channel" }. Again a negative number.

Confirm the chat ID

Paste the chat ID into a temporary file or note. You'll set it as `HERALD_TGRAM_CHAT_ID` in the next step.

If `getUpdates` returns an empty result: `[]` array, the bot hasn't received any messages yet. Send another one and retry. If you're using webhook mode (rare for setup), `getUpdates` returns 409 — see Troubleshooting.

Step 3 — Provide the credentials to Herald

Choose ONE of these two paths (or both — the resolution order in `OPERATOR_CREDENTIALS.md` handles overlap correctly).

Path A — Shell-export (recommended for personal workstation)

Add to your `~/.zshrc` (macOS default) or `~/.bashrc` (Linux default):

```
# Herald — Telegram (HRD-011)
export HERALD_TGRAM_BOT_TOKEN='1234567890:AAH1lab2c3d4e5f6g7h8i9j0kK_LmN3oPqR4S5t'
export HERALD_TGRAM_CHAT_ID='-1001234567890' # or your DM chat ID
```

Then either run `source ~/.zshrc` (or `~/.bashrc`) or open a new terminal.

Verify:

```
echo "${HERALD_TGRAM_BOT_TOKEN:0:10}..." # should print the
first 10 chars
echo "${HERALD_TGRAM_CHAT_ID}"
# should print the chat ID
```

Path B — Project-local .env

Edit (or create) `/Users/milosvasic/Projects/Herald/.env`:

```
HERALD_TGRAM_BOT_TOKEN=1234567890:AAH1lab2c3d4e5f6g7h8i9j0kK_LmN3oPqR4S5t
HERALD_TGRAM_CHAT_ID=-1001234567890
```

Set restrictive permissions:

```
chmod 600 .env
```

For native pherald (NOT through compose), source it into your shell first:

```
set -a; source .env; set +a
```

For compose runs, `docker-compose` reads `.env` automatically.

Per §11.4.10: never `git add .env`. The repo's `.gitignore` already covers it (line 28), but always double-check before committing.

Step 4 — Verify HealthCheck

The `HealthCheck` integration test issues a real `getMe` call against the Bot API. It proves the token is valid and the bot is enabled.

```
cd /Users/milosvasic/Projects/Herald
go test ./commons_messaging/channels/tgram/ -tags=integration
-run TestHealthCheck_LiveBotAPI -count=1 -timeout=60s
```

Expected:

```
--- PASS: TestHealthCheck_LiveBotAPI (1.x s)
PASS
```

If you see `SKIP`, the env vars aren't visible to `go test`. Re-source your shell or run the command with the env vars inline:

```
HERALD_TGRAM_BOT_TOKEN='...' HERALD_TGRAM_CHAT_ID='...' \
go test ./commons_messaging/channels/tgram/ -tags=integration
-run TestHealthCheck_LiveBotAPI -count=1 -timeout=60s
```

If you see `FAIL` with `getMe` returned empty body or HTTP errors, the token is invalid. Re-check Step 1.

Step 5 — Verify Send (E17)

The E17 evidence test sends a real Telegram message to your configured chat AND persists a row in the `outbound_delivery_evidence` table. It proves the full Send + persistence round-trip works.

Pre-requisite: container runtime (podman or docker) available. The test boots a Postgres container automatically.

```
cd /Users/milosvasic/Projects/Herald
go test ./commons_messaging/channels/tgram/ -tags=integration
    -run TestSend_PersistsDeliveryEvidence -count=1
    -timeout=300s -v
```

Expected:

```
--- PASS: TestSend_PersistsDeliveryEvidence (Xs)
PASS
```

You should also see the test message arrive in your Telegram chat — something like:

```
> Herald E17 persist test 2026-05-21T11:23:45.789Z
```

The §107 anti-bluff guard asserts the persisted `channel_message_id` equals the Telegram-side `MessageID` exactly — not a Herald-generated UUID. A spoofed adapter that returned a fake ID would fail this test.

Step 6 — (Optional) Verify Subscribe + Vertical Slice (E19)

E19 is the canonical Telegram → Claude Code → Telegram round-trip. It requires:

1. All of Step 3's env vars set.
2. `HERALD_CLAUDE_BIN` resolves to a working `claude` CLI (default: looked up via `$PATH`).
3. `HERALD_CLAUDE_PROJECT_NAME` set (see [../dispatchers/CLAUDE_CODE.md](#)).
4. `HERALD_TGRAM_LIVE_INBOUND=1` to enable the live-inbound code path.
5. **You hand-send a Telegram message to the bot during the test window** (60s default).

```
HERALD_TGRAM_LIVE_INBOUND=1 \
go test ./commons_messaging/ -tags=integration -run
    TestVerticalSlice_TelegramClaudeRoundTrip -count=1
    -timeout=300s -v
```

Within the 60s window: open Telegram → send any message to your bot. The handler will (a) receive your message, (b) dispatch it to Claude Code, (c) parse the reply, (d) send the reply back to your chat.

Expected:

```
E19 PASS: tenant=...; inbound=1; dispatch ok; outbound ok
(channel_id=...)
--- PASS: TestVerticalSlice_TelegramClaudeRoundTrip (Xs)
```

You should see TWO messages in your Telegram chat: yours, then Claude’s reply.

Step 7 — (Future) Webhook ingress for production

Long-poll (Steps 1-6) is what Herald uses today. For lower latency in production, Telegram supports webhook ingress — Telegram POSTs each update to your endpoint.

Status: NOT YET IMPLEMENTED — reserved for HRD-NNN.

When webhook ingress lands, you’ll:

1. Generate a 256-bit hex secret:

```
openssl rand -hex 32
```

2. Set `HERALD_TGRAM_WEBHOOK_SECRET=<the-hex>`.

3. Configure the bot’s webhook URL via the Bot API:

```
https://api.telegram.org/bot<TOKEN>/setWebhook?url=https://  
your-herald.example.com/v1/webhooks/tgram&secret_token=<the-  
hex>
```

4. Telegram POSTs each update to `/v1/webhooks/tgram` with the secret in the `X-Telegram-Bot-API-Secret-Token` header for verification.

Until webhook ingress lands, leave `HERALD_TGRAM_WEBHOOK_SECRET` unset and rely on long-poll.

Troubleshooting

“401 Unauthorized” from getMe: Token is wrong. Re-copy from BotFather; check no trailing whitespace.

getUpdates returns 409 Conflict: A webhook is configured for this bot. Either use the webhook path (Step 7) OR delete the webhook to switch back to long-poll:

```
https://api.telegram.org/bot<TOKEN>/deleteWebhook
```

Bot doesn’t see group messages: For groups, you must DISABLE “Privacy Mode” in BotFather: 1. Chat with @BotFather → /mybots → select your bot → “Bot Settings” → “Group Privacy” → “Turn off”. 2. Re-add the bot to the group if it was already added (the privacy change takes effect for new sessions).

Test SKIPS despite env vars set: Some test runners scope env vars per-process. Run with inline env: `HERALD_TGRAM_BOT_TOKEN='...' go test ...`

MarkdownV2 parse error: Herald uses MarkdownV2 parse mode by default (spec §11.1). Telegram requires specific escaping for `_`, `*`, `[`, `]`, `(`, `)`, `~`, ```, `>`, `#`, `+`, `-`, `=`, `|`, `{`, `}`, `.`, `!`. If a message contains unescaped versions, the Send returns 400. Use `commons.Body.Plain` for plain-text without parse mode interpretation.

“chat not found”: The bot was removed from the group, or you’re using the wrong chat ID. Re-run `getUpdates` to confirm the chat exists.

Spec + code references

- **Spec**: V3 §11.1 (Telegram capabilities + Bot API surface), §32.2 (subscriber-reply long-poll loop 25s + 30s safety-net)
- **HRD**: HRD-011 (Telegram channel adapter live integration)
- **Code**:
 - `commons_messaging/channels/tgram/tgram.go` — Adapter struct + URL parser + Capabilities
 - `commons_messaging/channels/tgram/healthcheck.go` — `getMe HealthCheck`
 - `commons_messaging/channels/tgram/send.go` — `Bot.Send (sendMessage MarkdownV2 + forum-topic ThreadID)`
 - `commons_messaging/channels/tgram/subscribe.go` — `LongPoller 25s + 30s safety-net per §32.2`
 - `commons_messaging/channels/tgram/persist.go` — `NewWithStorage + SendForTenant (outbound_delivery_evidence persistence)`
- **Tests**:
 - `commons_messaging/channels/tgram/healthcheck_integration_test.go` — E17 prerequisite
 - `commons_messaging/channels/tgram/send_integration_test.go` — E17 (live Telegram + non-empty `Receipt.ChannelMsgID`)
 - `commons_messaging/channels/tgram/subscribe_integration_test.go` — E19 inbound half
 - `commons_messaging/channels/tgram/persist_integration_test.go` — E17 persistence (exact `channel_message_id` match)
 - `commons_messaging/vertical_slice_integration_test.go` — E19 full slice
- **Vendored SDK**: `submodules/telebot/` (`gopkg.in/telebot.v3` at v3.3.8, pinned to keep `gopkg.in/telebot.v3` import-path stable per §11.4.74)
- **Related**: `OPERATOR_CREDENTIALS.md` §“Telegram”; `../dispatchers/CLAUDE_CODE.md` for the LLM dispatch half of the vertical slice